AI-ASSISTED CLINICAL TRIAGE

Lung Histopathology Triage System

An explainable deep-learning classifier that triages lung tissue slides into adenocarcinoma, squamous cell carcinoma, or benign - with visual evidence a pathologist can inspect.

3-Class Classification | Grad-CAM Explainability | FastAPI + React | Model v4.1.0

THE PROBLEM

Pathology triage is a bottleneck

Every lung biopsy slide must be reviewed by a specialist. Demand grows faster than the pathologist workforce, and the first question on every desk is the same: **which cases need attention first?**

Volume

Slide caseloads rise year on year while specialist numbers stay flat. Backlogs delay diagnosis.

Priority

Malignant and benign cases sit in the same queue. There is no fast first-pass signal to sort them.

Trust

A model that only outputs a label is not clinically useful. Reviewers need to see *why* it decided.

THE SOLUTION

A triage layer, not a replacement

Upload a slide, receive a class prediction with a confidence score and a Grad-CAM heatmap highlighting the tissue regions that drove the decision. The pathologist reviews the evidence and makes the call.

Upload
Histopathology image (JPG/PNG)

→

Classify
ResNet backbone, 3-class head

→

Explain
Grad-CAM activation overlay

→

Review
Clinician inspects and decides

Human in the loop. The system produces decision support only. No diagnosis is issued without a qualified pathologist reviewing the output.

KEY FEATURES

What the system does

Three-Class Triage

Separates adenocarcinoma, squamous cell carcinoma, and benign tissue in a single pass.

Grad-CAM Explainability

Returns a heatmap showing which regions of the slide influenced the prediction.

Confidence Scoring

Per-class probability distribution, so low-confidence cases can be flagged for review.

Batch Processing

Submit multiple slides in one request for high-throughput screening runs.

Clinical Dashboard

React interface with upload zone, side-by-side original vs heatmap viewer, and live stats.

REST API + OpenAPI

Typed JSON endpoints with auto-generated interactive docs at `/docs`.

FEATURE 01

Three-class prediction

A single POST request returns the predicted class, the full probability distribution, and the model version that produced it.

- **Adenocarcinoma** - malignant glandular tissue
- **Squamous cell carcinoma** - malignant squamous tissue
- **Benign** - non-malignant lung tissue

REQUEST

```
POST /predict
Content-Type: multipart/form-data

file: slide image
```

RESPONSE

```
{
  "prediction": "benign",
  "confidence": 0.915,
  "probabilities": {
    "adenocarcinoma": 0.041,
    "squamous_cell_carcinoma": 0.044,
    "benign": 0.915
  },
  "model_version": "4.1.0",
  "heatmap": "base64 PNG"
}
```

Real output: **benign - 91.5%** on a supplied tissue image.

FEATURE 02

Grad-CAM explainability

A confidence score alone does not earn clinical trust. The system back-propagates the winning class through the final convolutional block to produce a heatmap of the pixels that mattered most.

Forward pass
Image through ResNet backbone

→

Hook
Capture gradients at layer4

→

Weight
Channel activations by gradient

→

Overlay
Colormap blended onto original

For the reviewer

Confirms the model is looking at tissue morphology rather than slide artefacts, pen marks, or background.

For the auditor

Every prediction ships with its evidence. Disagreements can be examined rather than argued.

FEATURE 03

The clinical dashboard

A React interface built for pathologists, not for engineers. Drag a slide in, read the result, inspect the heatmap, export the report.

1

Upload Zone

Drag-and-drop or browse. Validates file type and shows upload progress.

2

Heatmap Viewer

Tabbed viewer switching between the original slide and the Grad-CAM overlay.

3

Stats Panel

Predicted class, confidence bar, and probability breakdown per class.

React + Vite | Tailwind CSS | shadcn/ui components | NHS-branded header | Print / export report | Responsive layout

FEATURE 04

API surface

ENDPOINTS

GET /health

Liveness and model status. Returns device, model version, and class count. Used by the platform health check.

POST /predict

Single-image classification with heatmap generation.

POST /predict/batch

Multiple images in one request for bulk screening.

GET /docs

Auto-generated OpenAPI reference, interactive and always current.

LIVE HEALTH RESPONSE

```
{
  "status": "live",
  "device": "cpu",
  "model_version": "4.1.0",
  "num_classes": 3
}
```

Verified running locally. The backend answers on port 8000 with the model loaded and ready for inference.

START IT

```
.venv\Scripts\python.exe -m uvicorn api:app --port 8000
```

ARCHITECTURE

How the pieces fit

React Dashboard
Vite build, static assets

→

REST / JSON
multipart upload, base64 response

→

FastAPI Service
api.py, lifespan model load

→

PyTorch Model
ResNet backbone, 3-class head

Frontend

Static React bundle. Deploys to any CDN - Vercel, Netlify, or served by the API itself.

Backend

Long-running Python process holding the model in memory. Needs a container host, not a serverless function.

Model

92 MB checkpoint loaded once at startup. CPU inference supported; GPU optional.

Why not serverless? Model weights must stay resident. A cold start per request would reload 92 MB of tensors and destroy response time.

MACHINE LEARNING

Model and training

ARCHITECTURE

- **Backbone** - ResNet50, ImageNet pre-trained
- **Head** - 3-class linear classifier, dropout 0.3
- **Input** - 224 x 224 RGB tiles
- **Dataset** - LC25000 histopathological image set

TRAINING STRATEGY

- **Phase 1** - frozen backbone, head only, higher LR
- **Phase 2** - full fine-tune, LR 1e-4, plateau scheduler

CONFIGURATION

```
model:  resnet50
classes: 3
dropout: 0.3
img_size: 224
batch_size: 32
epochs: 20
lr: 1.0e-4
scheduler: plateau
```

Two-phase training protects the pre-trained features early, then adapts the whole network to histopathology once the head has stabilised.

DEPLOYMENT

Running it in production

Backend - container host

Railway via the included `railway.toml`. Nixpacks builds the Python environment, the health check polls `/health`, and on-failure restarts are capped at three.

Frontend - static CDN

Vercel or any static host. Run the Vite build, publish the output directory, point the API base URL at the backend.

ENVIRONMENT VARIABLES

```
PORT=8000
MODEL_PATH=model.pth
```

HEALTH CHECK

```
healthcheckPath = "/health"
healthcheckTimeout = 100
restartPolicyType = "on-failure"
```

Weights are not in the repo. Model files are gitignored to stay under hosting limits - the container must fetch them at build time.

HONEST STATUS

What is proven, and what is not

WORKING AND VERIFIED

- FastAPI backend runs and serves `/health` with the model loaded
- Grad-CAM heatmaps generate end-to-end
- Dashboard upload-to-result flow connected to the API
- Two images classified end-to-end with real outputs
- Deployment configuration written for both services

NOT YET ESTABLISHED

- Full LC25000 dataset not downloaded - no train/validation run in this workspace
- No accuracy, sensitivity, or specificity figures measured
- Only two test images; validation set absent
- One image labelled adenocarcinoma was predicted benign at 91.5% - unexplained
- Legacy `backend/main.py` mismatches the checkpoint and fails to load

Read this slide aloud. The system is a tested pipeline, not a validated clinical tool. Any performance claim must wait for a proper held-out evaluation.

NEXT STEPS

Path to a validated tool

1

Acquire the dataset

Run the LC25000 downloader and materialise the processed train/validation split.

2

Train and evaluate

Execute the two-phase schedule and record accuracy, sensitivity, specificity, and confusion matrix per class.

3

Resolve the discrepancy

Investigate the mispredicted adenocarcinoma case before making any accuracy claim.

4

Harden and deploy

Remove the stale backend, wire the dashboard to the deployed API, and run a pilot review.

SUMMARY

Explainable triage, built for review

Three-class lung tissue classification, Grad-CAM evidence on every prediction, and a clinical dashboard a pathologist can actually use. The pipeline runs today; the validation is the next milestone.

Backend live on port 8000

Dashboard ready

Deployment configured